

Name: Irtiza Ali

Internship Domain: Data Science

Task Number: 03

Technologies Used: Python, Pandas, SQL, Power BI and Google Colab.

Dataset Description:

The dataset contains 9,994 retail sales records with 32 columns. It includes information about orders, customers, products, regions, sales, quantity, discounts, and profit. The data covers transactions from 2014 to 2017. Additional features such as shipping days, profit margin, sales per unit, profit status, and discount bands were also created during preprocessing to support business analysis and dashboard development.

Dataset Source: <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final?resource=download>

GitHub Repository Link:

<https://github.com/irtizaali283/superstore-business-intelligence-dashboard.git>

15 Business Insights:

1. Total sales reached about \$2.30M.
2. Total profit was about \$286.4K.
3. The business handled around 5,009 orders from 793 customers.
4. Overall profit margin was 12.47%.
5. 2017 had the highest yearly sales, around \$733K.
6. The West region generated the highest sales, around \$725K.
7. West also generated the highest regional profit, around \$108K.
8. Technology was the top category with about \$836K in sales.
9. Technology also produced the highest category profit, around \$145K.
10. Furniture generated high sales but very low profit, only about \$18K.
11. Consumer customers generated the largest share of sales.
12. Phones were the highest-selling sub-category.
13. Tables made strong sales but produced a loss of about \$17.7K.
14. Sales without discounts produced the highest profit, while medium and high discounts caused overall losses.
15. The top-selling customer was Sean Miller, but his transactions resulted in an overall loss, showing that high sales do not always mean high profit.

10 Business Recommendations:

1. Focus more on Technology and Office Supplies, because they generate strong profits.
2. Review pricing and costs for the Furniture category.
3. Investigate why Tables are generating losses.
4. Reduce unnecessary high and medium discounts.
5. Give discounts only where they help increase profitable sales.
6. Continue investing in the West region, while also improving weaker regions.
7. Promote high-performing products such as Phones and Copiers.
8. Monitor customers with high sales but negative profit.
9. Track profit along with sales instead of focusing only on revenue.
10. Review the dashboard regularly to identify weak products, regions, and discount strategies early.

Executive Summary:

The Superstore business generated about \$2.30M in sales and \$286.4K in profit, with an overall profit margin of 12.47%. Sales improved over time, and 2017 was the strongest year. Technology and Office Supplies performed well, while Furniture had much lower profitability. High discounts were also a major reason for losses.

SQL ANALYSIS:

```
CREATE DATABASE SuperStoreDB;
```

```
USE SuperstoreDB;
```

```
--How many rows are in the table?
```

```
SELECT COUNT(*) AS total_rows  
FROM dbo.superstore_sales;
```

```
--TOP 10 rows of the table
```

```
SELECT TOP 10 *  
FROM dbo.superstore_sales;
```

```
--How much total sales did the company generate?
```

```
SELECT SUM(sales) AS total_sales  
FROM dbo.superstore_sales;
```

--How much total profit did the company make overall?

```
SELECT SUM(profit) AS total_profit
FROM dbo.superstore_sales;
```

--How much revenue did the business make each month?

```
SELECT year_month, SUM(sales) AS monthly_revenue
FROM dbo.superstore_sales
GROUP BY year_month
ORDER BY year_month;
```

--Which products generated the most sales?

```
SELECT TOP 10
    product_name,
    SUM(sales) AS total_sales
FROM dbo.superstore_sales
GROUP BY product_name
ORDER BY total_sales DESC;
```

--Which customers generated the most sales?

```
SELECT TOP 10
    customer_name,
    SUM(sales) AS total_sales
FROM dbo.superstore_sales
GROUP BY customer_name
ORDER BY total_sales DESC;
```

--Which regions perform best in sales and profit?

```
SELECT
    region,
    SUM(sales) AS total_sales,
    SUM(profit) AS total_profit
FROM dbo.superstore_sales
GROUP BY region
ORDER BY total_sales DESC;
```

--Which category generates the most sales and profit?

--Which category generates the most sales and profit?

```
SELECT
    category,
    SUM(sales) AS total_sales,
    SUM(profit) AS total_profit
FROM dbo.superstore_sales
GROUP BY category
ORDER BY total_sales DESC;
```

--How much profit and loss did the business generate?

```
SELECT
    profit_status,
    SUM(profit) AS total_profit
FROM dbo.superstore_sales
GROUP BY profit_status;
```

--Are sales increasing or decreasing over the years?

```
SELECT
    order_year,
    SUM(sales) AS total_sales
FROM dbo.superstore_sales
GROUP BY order_year
ORDER BY order_year;
```

--Which sub-categories perform best in sales and profit?

```
SELECT
    sub_category,
    SUM(sales) AS total_sales,
    SUM(profit) AS total_profit
FROM dbo.superstore_sales
GROUP BY sub_category
ORDER BY total_sales DESC;
```

--Which discount levels generate sales, and which ones may reduce profit?

```
SELECT
    discount_band,
    SUM(sales) AS total_sales,
    SUM(profit) AS total_profit
FROM dbo.superstore_sales
GROUP BY discount_band
ORDER BY total_sales DESC;
```

Preprocessing:

Imports Pandas and NumPy, which are used to work with and analyze the dataset.

```
import pandas as pd
import numpy as np
```

Uploads the Sample - Superstore file from the computer and loads it into Colab.

```
from google.colab import files
uploaded = files.upload()
df = pd.read_csv('Sample - Superstore.csv', encoding='cp1252')
```

Sample - Superstore.csv
Sample: Superstore.csv(uploaded) - 2287800 bytes, last modified: 9/29/2016 - 100% done
Saving Sample - Superstore.csv to Sample - Superstore.csv

Displays the first five rows to get a quick look at the dataset.

```
df.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
0	1	CA-2016-152156	11/9/2016	11/11/2016	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	...	42420	South	FUR-BO-10001796	Furniture	Bookcases	Bush Somerset Collection Bookcase	201.9000	2	0.00	41.9136
1	2	CA-2016-152156	11/9/2016	11/11/2016	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000494	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400	3	0.00	219.5820
2	3	CA-2016-138988	9/12/2016	9/19/2016	Second Class	DV-13045	Darin Van Huff	Corporate	United States	Los Angeles	...	90036	West	OFF-LA-10000240	Office Supplies	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200	2	0.00	6.8744
3	4	US-2016-106998	10/11/2016	10/18/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	FUR-TA-10000577	Furniture	Tables	Bradford CR4500 Series Slim Rectangular Table	967.5775	6	0.45	383.0310
4	5	US-2016-106998	10/11/2016	10/18/2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	South	OFF-ST-10000780	Office Supplies	Storage	Ekorn Fold 'N Roll Cart System	22.3080	2	0.20	2.5104

5 rows x 21 columns

Displays the last five rows to get a quick look at the dataset.

```
df.tail()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
9990	9990	CA-2014-110422	1/21/2014	1/23/2014	Second Class	TB-21400	Tom Brockenhuuser	Consumer	United States	Miami	...	33180	South	FUR-FU-10001389	Furniture	Furnishings	Ultra Door Pull Handle	25.248	3	0.2	4.1028
9990	9991	CA-2017-121258	2/26/2017	3/2/2017	Standard Class	DB-13080	Dave Brooks	Consumer	United States	Costa Mesa	...	92627	West	FUR-FU-10000747	Furniture	Furnishings	Tenex B1-RE Series Chair Mats for Low Pile Car...	91.900	2	0.0	15.6332
9991	9992	CA-2017-121258	2/26/2017	3/2/2017	Standard Class	DB-13080	Dave Brooks	Consumer	United States	Costa Mesa	...	92627	West	TEC-PI-10003645	Technology	Phones	Asstra S7i VoIP phone	258.576	2	0.2	19.3632
9992	9993	CA-2017-121258	2/26/2017	3/2/2017	Standard Class	DB-13080	Dave Brooks	Consumer	United States	Costa Mesa	...	92627	West	OFF-PA-10004041	Office Supplies	Paper	It's Hot Message Books with Stickers, 2 3/4" x 9"	20.800	4	0.0	13.3200
9993	9994	CA-2017-110914	5/4/2017	5/9/2017	Second Class	CG-12220	Chris Cortes	Consumer	United States	Westminster	...	62683	West	OFF-AP-10002884	Office Supplies	Appliances	Acico 7-Outlet Montpelier Power Center, Withou...	243.160	2	0.0	72.9480

5 rows x 21 columns

Checks how many rows and columns are present in the data.

```
df.shape
```

(9994, 21)

Lists all the column names available in the dataset.

```
df.columns
```

Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
 'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',
 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
 'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'],
 dtype='object')

Gives an overview of the columns, data types, and non-empty values.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Row ID          9994 non-null   int64
 1   Order ID        9994 non-null   object
 2   Order Date      9994 non-null   object
 3   Ship Date       9994 non-null   object
 4   Ship Mode       9994 non-null   object
 5   Customer ID     9994 non-null   object
 6   Customer Name   9994 non-null   object
 7   Segment         9994 non-null   object
 8   Country         9994 non-null   object
 9   City           9994 non-null   object
10   State          9994 non-null   object
11   Postal Code     9994 non-null   int64
12   Region         9994 non-null   object
13   Product ID     9994 non-null   object
14   Category       9994 non-null   object
15   Sub-Category   9994 non-null   object
16   Product Name    9994 non-null   object
17   Sales          9994 non-null   float64
18   Quantity       9994 non-null   int64
19   Discount       9994 non-null   float64
20   Profit         9994 non-null   float64
dtypes: float64(3), int64(3), object(15)
memory usage: 1.6+ MB
```

✓ Checks each column for missing values.

```
56) df.isnull().sum()
```

```
57) *** 0
      Row ID  0
      Order ID  0
      Order Date  0
      Ship Date  0
      Ship Mode  0
      Customer ID  0
      Customer Name  0
      Segment  0
      Country  0
      City  0
      State  0
      Postal Code  0
      Region  0
      Product ID  0
      Category  0
      Sub-Category  0
      Product Name  0
      Sales  0
      Quantity  0
      Discount  0
      Profit  0
```

dtype: int64

✓ Counts whether any complete rows are duplicated.

```
57) df.duplicated().sum()
```

```
58) np.int64(0)
```

Summarizes the numerical data with values such as average, minimum, and maximum.

```
[18] df.describe()
```

	Row ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	55190.379428	229.858001	3.789574	0.156203	28.856896
std	2885.163629	32063.693350	623.245101	2.225110	0.206452	234.260108
min	1.000000	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2499.250000	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	4997.500000	58430.500000	54.490000	3.000000	0.200000	8.868500
75%	7495.750000	90008.000000	209.940000	5.000000	0.200000	29.364000
max	9994.000000	99301.000000	22638.480000	14.000000	0.800000	8399.978000

Finds the text-based columns and counts how many different values each one contains.

```
[19] categorical_columns = df.select_dtypes(include="object").columns
df[categorical_columns].nunique()
```

	0
Order ID	5009
Order Date	1237
Ship Date	1334
Ship Mode	4
Customer ID	793
Customer Name	793
Segment	3
Country	1
City	531
State	49
Region	4
Product ID	1882
Category	3
Sub-Category	17
Product Name	1850

dtype: int64

▼ Displays the actual values of text columns that have a small number of categories.

```
(00)
✓ Out for col in categorical_columns:
      if df[col].nunique() <= 20:
          print(f"{col}:")
          print(df[col].unique())
          print()

▼
Ship Mode:
['Second Class' 'Standard Class' 'First Class' 'Same Day']

Segment:
['Consumer' 'Corporate' 'Home Office']

Country:
['United States']

Region:
['South' 'West' 'Central' 'East']

Category:
['Furniture' 'Office Supplies' 'Technology']

Sub-Category:
['Bookcases' 'Chairs' 'Labels' 'Tables' 'Storage' 'Furnishings' 'Art'
 'Phones' 'Binders' 'Appliances' 'Paper' 'Accessories' 'Envelopes'
 'Fasteners' 'Supplies' 'Machines' 'Copiers']
```

▼ Converts the order and shipping dates from text into proper date format.

```
(01)
✓ Out df["Order Date"] = pd.to_datetime(df["Order Date"], format="%m/%d/%Y")
df["Ship Date"] = pd.to_datetime(df["Ship Date"], format="%m/%d/%Y")
```

▼ Confirms that both date columns were successfully converted to date format.

```
(02)
✓ Out df[["Order Date", "Ship Date"]].dtypes

▼
      0
Order Date  datetime64[ns]
Ship Date   datetime64[ns]

dtype: object
```

✓ Makes the column names cleaner and easier to use by standardizing their format.

```
[63] ✓ 0s
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
    .str.replace("-", "_")
)

df.columns

Index(['row_id', 'order_id', 'order_date', 'ship_date', 'ship_mode',
       'customer_id', 'customer_name', 'segment', 'country', 'city', 'state',
       'postal_code', 'region', 'product_id', 'category', 'sub_category',
       'product_name', 'sales', 'quantity', 'discount', 'profit'],
      dtype='object')
```

✓ Cleans the text inside the rows by removing unwanted and extra spaces.

```
[64] ✓ 0s
text_columns = df.select_dtypes(include="object").columns

for col in text_columns:
    df[col] = (
        df[col]
        .str.replace("\xa0", " ", regex=False)
        .str.strip()
        .str.replace(r"\s+", " ", regex=True)
    )
```

✓ Reviews the cleaned category values to make sure the text looks consistent.

```
[65] ✓ 0s
for col in text_columns:
    if df[col].nunique() <= 20:
        print(f"{col}:")
        print(df[col].unique())
        print()

ship_mode:
['Second Class' 'Standard Class' 'First Class' 'Same Day']

segment:
['Consumer' 'Corporate' 'Home Office']

country:
['United States']

region:
['South' 'West' 'Central' 'East']

category:
['Furniture' 'Office Supplies' 'Technology']

sub_category:
['Bookcases' 'Chairs' 'Labels' 'Tables' 'Storage' 'Furnishings' 'Art'
 'Phones' 'Binders' 'Appliances' 'Paper' 'Accessories' 'Envelopes'
 'Fasteners' 'Supplies' 'Machines' 'Copiers']
```

✓ Checks for duplicate rows again after cleaning the text.

```
df.duplicated().sum()
np.int64(0)
```

✓ Looks for invalid sales, quantity, discount, and negative-profit values.

```
print("Sales <= 0:", (df["sales"] <= 0).sum())
print("Quantity <= 0:", (df["quantity"] <= 0).sum())
print("Discount < 0:", (df["discount"] < 0).sum())
print("Discount > 1:", (df["discount"] > 1).sum())
print("Negative Profit:", (df["profit"] < 0).sum())

Sales <= 0: 0
Quantity <= 0: 0
Discount < 0: 0
Discount > 1: 0
Negative Profit: 1871
```

✓ Makes sure no order was shipped before it was actually placed.

```
(df["ship_date"] < df["order_date"]).sum()
np.int64(0)
```

✓ Identifies unusually high or low numerical values using the IQR method.

```
numeric_cols = ["sales", "quantity", "discount", "profit"]

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = ((df[col] < lower_bound) | (df[col] > upper_bound)).sum()

    print(f"{col}: {outliers} potential outliers")

*** sales: 1167 potential outliers
    quantity: 170 potential outliers
    discount: 856 potential outliers
    profit: 1881 potential outliers
```

Shows the ten transactions with the highest sales for closer inspection.

```
df.nlargest(10, "sales")
[ "order_id", "customer_name", "product_name",
  "sales", "quantity", "discount", "profit" ]
```

	order_id	customer_name	product_name	sales	quantity	discount	profit
2697	CA-2014-145317	Sean Miller	Cisco TelePresence System EX90 Videoconferenci...	22638.480	6	0.5	-1811.0784
6826	CA-2016-118689	Tamara Chand	Canon imageCLASS 2200 Advanced Copier	17499.950	5	0.0	8399.9760
8153	CA-2017-140151	Raymond Buch	Canon imageCLASS 2200 Advanced Copier	13999.900	4	0.0	6719.9808
2623	CA-2017-127180	Tom Ashbrook	Canon imageCLASS 2200 Advanced Copier	11199.968	4	0.2	3919.9888
4190	CA-2017-166709	Hunter Lopez	Canon imageCLASS 2200 Advanced Copier	10499.970	3	0.0	5039.9856
9039	CA-2016-117121	Adrian Barton	GBC Ibimaster 500 Manual ProClick Binding System	9882.740	13	0.0	4948.3700
4098	CA-2014-118904	Sanjit Chand	Ibico EPK-21 Electric Binding System	9449.950	5	0.0	4830.4755
4277	US-2016-107440	Bill Shoney	3D Systems Cube Printer, 2nd Generation, Magenta	9099.930	7	0.0	2385.9818
8488	CA-2016-158841	Sanjit Engle	HP Designjet T520 Inkjet Large Format Printer ...	8749.950	5	0.0	2799.9840
6425	CA-2016-143714	Christopher Conant	Canon imageCLASS 2200 Advanced Copier	8399.976	4	0.4	1119.9968

Finds the ten transactions that caused the largest losses.

```
df.nsmallest(10, "profit")
[ "order_id", "customer_name", "product_name",
  "sales", "quantity", "discount", "profit" ]
```

	order_id	customer_name	product_name	sales	quantity	discount	profit
7772	CA-2016-108190	Cindy Stewart	Cubify CubeX 3D Printer Double Head Print	4499.985	5	0.7	-6599.9780
683	US-2017-168116	Grant Thornton	Cubify CubeX 3D Printer Triple Head Print	7999.980	4	0.5	-3839.9904
9774	CA-2014-169019	Luke Foster	GBC DocuBind P400 Electric Binding System	2177.584	8	0.8	-3701.8928
3011	CA-2017-134845	Sharelle Roach	Lexmark MX811dhe Monochrome Laser Printer	2549.985	5	0.7	-3399.9800
4991	US-2017-122714	Henry Goldwyn	Ibico EPK-21 Electric Binding System	1889.990	5	0.8	-2629.4845
3151	CA-2015-147830	Natalie Fritzer	Cubify CubeX 3D Printer Double Head Print	1799.994	2	0.7	-2639.9912
5310	CA-2017-131254	Nathan Cano	Fellowes PB500 Electric Punch Plastic Comb Bin...	1525.188	6	0.8	-2287.7820
9639	CA-2015-118638	Joseph Holt	Chromcraft Bull-Nose Wood Oval Conference Tabl...	4297.844	13	0.4	-1862.3124
1199	CA-2016-130946	Zuschuss Carroll	GBC DocuBind P400 Electric Binding System	1088.792	4	0.8	-1850.9464
2697	CA-2014-145317	Sean Miller	Cisco TelePresence System EX90 Videoconferenci...	22638.480	6	0.5	-1811.0784

Displays the ten transactions that generated the highest profits.

[72]
✓ Oa

```
df.nlargest(10, "profit")[
    ["order_id", "customer_name", "product_name",
     "sales", "quantity", "discount", "profit"]
]
```

▼

	order_id	customer_name	product_name	sales	quantity	discount	profit	
6826	CA-2016-118889	Tamara Chand	Canon imageCLASS 2200 Advanced Copier	17499.950	5	0.0	8399.9780	
8153	CA-2017-140151	Raymond Buch	Canon imageCLASS 2200 Advanced Copier	13999.980	4	0.0	6719.9808	
4190	CA-2017-168709	Hunter Lopez	Canon imageCLASS 2200 Advanced Copier	10499.970	3	0.0	5039.9858	
9039	CA-2016-117121	Adrian Barton	GBC Ibmaster 500 Manual ProClick Binding System	9892.740	13	0.0	4948.3700	
4098	CA-2014-116904	Sanjit Chand	Ibico EPK-21 Electric Binding System	9449.950	5	0.0	4630.4755	
2623	CA-2017-127180	Tom Ashbrook	Canon imageCLASS 2200 Advanced Copier	11199.968	4	0.2	3919.9888	
509	CA-2015-145352	Christopher Martinez	Fellowes PB500 Electric Punch Plastic Comb Bin...	6354.950	5	0.0	3177.4750	
8488	CA-2016-158841	Sanjit Engle	HP Designjet T520 Inkjet Large Format Printer ...	8749.950	5	0.0	2799.9840	
7666	US-2016-140158	Daniel Raglin	Hewlett Packard LaserJet 3310 Copier	5399.910	9	0.0	2591.9568	
6520	CA-2017-138289	Andy Reiter	GBC DocuBind P400 Electric Binding System	5443.980	4	0.0	2504.2216	

Reviews the transactions where the largest quantities of products were purchased.

[73]
✓ Oa

```
df.nlargest(10, "quantity")[
    ["order_id", "product_name", "sales",
     "quantity", "discount", "profit"]
]
```

▼

	order_id	product_name	sales	quantity	discount	profit	
113	CA-2014-115259	OIC Colored Binder Clips, Assorted Sizes	40.096	14	0.2	14.5348	
139	CA-2016-145583	Longer-Life Soft White Bulbs	43.120	14	0.0	20.6976	
575	CA-2015-149713	Personal Creations Ink Jet Cards and Labels	180.720	14	0.0	78.7528	
660	CA-2015-148563	Space Solutions Commercial Steel Shelving	724.080	14	0.2	-135.7650	
1045	CA-2017-152702	Global Stack Chair without Arms, Black	254.604	14	0.3	-18.1860	
1363	US-2017-155425	Cardinal EasyOpen D-Ring Binders	38.388	14	0.7	-25.5920	
1429	US-2015-164448	Avery 4027 File Folder Labels for Dot Matrix P...	427.420	14	0.0	196.6132	
1433	CA-2014-120768	High-Back Leather Manager's Chair	1819.860	14	0.0	163.7874	
1711	CA-2017-123491	Acco 7-Outlet Masterpiece Power Center, Whitou...	1702.120	14	0.0	510.6360	
2793	CA-2014-154599	Pyle PMP37LED	1075.088	14	0.2	94.0702	

Shows each discount level and how many transactions used that discount.

```
df["discount"].value_counts().sort_index()
```

discount	count
0.00	4798
0.10	94
0.15	52
0.20	3857
0.30	227
0.32	27
0.40	206
0.45	11
0.50	66
0.60	138
0.70	418
0.80	300

dtype: int64

Checks whether product IDs and product names are consistently linked to each other.

```
product_id_check = df.groupby("product_id")["product_name"].nunique()
product_name_check = df.groupby("product_name")["product_id"].nunique()

print("Product IDs linked to multiple names:",
      (product_id_check > 1).sum())

print("Product names linked to multiple IDs:",
      (product_name_check > 1).sum())
```

Product IDs linked to multiple names: 32
Product names linked to multiple IDs: 16

+ Code

Displays the product IDs that are connected to more than one product name.

```
problem_product_ids = product_id_check[product_id_check > 1].index

df[df["product_id"].isin(problem_product_ids)][
    ["product_id", "product_name"]
].drop_duplicates().sort_values("product_id")
```

product_id	product_name
2471 FUR-BO-10002213	Sauder Forest Hills Library, Woodland Oak Finish
2115 FUR-BO-10002213	DMII Eclipse Executive Suite Bookcases
66 FUR-CH-10001146	Global Value Mid-Back Manager's Chair, Gray

Shows product names that appear under more than one product ID.

```
problem_product_names = product_name_check[product_name_check > 1].index
```

```
df[df["product_name"].isin(problem_product_names)][  
    ["product_name", "product_id"]  
].drop_duplicates().sort_values("product_name")
```

4063	Prang Drawing Pencil Set	OFF-AK-10002708
1556	Staple envelope	OFF-EN-10003134
1880	Staple envelope	OFF-EN-10002821
458	Staple envelope	OFF-EN-10001415
2911	Staple envelope	OFF-EN-10003286
3186	Staple envelope	OFF-EN-10004773
162	Staple envelope	OFF-EN-10001990
3936	Staple envelope	OFF-EN-10001028
896	Staple envelope	OFF-EN-10001099
1125	Staple envelope	OFF-EN-10001539
2284	Staple holder	OFF-AP-10002222
404	Staple holder	OFF-AP-10004249
4240	Staple holder	OFF-AP-10001366
857	Staple magnet	OFF-ST-10003221
1921	Staple magnet	OFF-ST-10002582
550	Staple remover	OFF-SU-10001935
2866	Staple remover	OFF-SU-10000946
367	Staple remover	OFF-SU-10001225
952	Staple-based wall hangings	FUR-FU-10001940
239	Staple-based wall hangings	FUR-FU-10004351
132	Staples	OFF-FA-10002780
1108	Staples	OFF-FA-10003495
4958	Staples	OFF-FA-10003485
2059	Staples	OFF-FA-10002975
1703	Staples	OFF-FA-10001843
1485	Staples	OFF-FA-10003021
525	Staples	OFF-FA-10002815
646	Staples	OFF-FA-10003112
2387	Staples	OFF-FA-10001229
1147	Staples	OFF-FA-10000735
7484	Staples in misc. colors	OFF-AR-10004707
2383	Staples in misc. colors	OFF-AR-10003087

Uses the most frequent Product Name and Product ID matches to create a standardized product ID.

Uses the most frequent Product Name and Product ID matches to create a standardized product ID.

```
name_id_freq = (
    df.groupby(["product_name", "product_id"])
      .size()
      .reset_index(name="frequency")
)

multi_id_names = (
    df.groupby("product_name")["product_id"]
      .nunique()
)

multi_id_names = multi_id_names[multi_id_names > 1].index

problem_pairs = name_id_freq[
    name_id_freq["product_name"].isin(multi_id_names)
].copy()

problem_pairs["max_frequency"] = (
    problem_pairs.groupby("product_name")["frequency"]
      .transform("max")
)

top_matches = problem_pairs[
    problem_pairs["frequency"] == problem_pairs["max_frequency"]
]

winner_count = (
    top_matches.groupby("product_name")["product_id"]
      .nunique()
)

resolved_names = winner_count[winner_count == 1].index
tied_names = winner_count[winner_count > 1].index

best_id_map = (
    top_matches[top_matches["product_name"].isin(resolved_names)]
      .set_index("product_name")["product_id"]
      .to_dict()
)

df["standard_product_id"] = df["product_id"]

mask = df["product_name"].isin(resolved_names)

df.loc[mask, "standard_product_id"] = (
    df.loc[mask, "product_name"].map(best_id_map)
)

print("Product names with multiple IDs:", len(multi_id_names))
print("Successfully standardized:", len(resolved_names))
print("Unresolved ties:", len(tied_names))
```

```
Product names with multiple IDs: 16
Successfully standardized: 13
Unresolved ties: 3
```

Resolves equal-frequency ID cases using a fixed rule and checks that the mappings

Resolves equal-frequency ID cases using a fixed rule and checks that the mappings are standardized.

```
tie_id_map = (
    top_matches[top_matches["product_name"].isin(tied_names)]
    .sort_values(["product_name", "product_id"])
    .drop_duplicates("product_name")
    .set_index("product_name")["product_id"]
    .to_dict()
)

# Apply chosen IDs to tied products
tie_mask = df["product_name"].isin(tied_names)

df.loc[tie_mask, "standard_product_id"] = (
    df.loc[tie_mask, "product_name"].map(tie_id_map)
)

final_check = (
    df[df["product_name"].isin(multi_id_names)]
    .groupby("product_name")["standard_product_id"]
    .nunique()
)

print("Product names with multiple IDs:", len(multi_id_names))
print("Successfully standardized:", (final_check == 1).sum())
print("Still unresolved:", (final_check > 1).sum())
```

```
--- Product names with multiple IDs: 16
    Successfully standardized: 16
    Still unresolved: 0
```

Creates unique standardized IDs for cases where one original ID belongs to different product names.

```
problem_ids = (
    df.groupby("product_id")["product_name"]
    .nunique()
)

problem_ids = problem_ids[problem_ids > 1].index

mask = df["product_id"].isin(problem_ids)

df.loc[mask, "standard_product_id"] = (
    df.loc[mask, "product_id"]
    + " | "
    + df.loc[mask, "product_name"]
)
```

Verifies that each standardized product ID now matches only one product name and vice versa.

```
id_name_check = df.groupby("standard_product_id")["product_name"].nunique()
name_id_check = df.groupby("product_name")["standard_product_id"].nunique()

print("Product IDs linked to multiple names:", (id_name_check > 1).sum())
print("Product names linked to multiple IDs:", (name_id_check > 1).sum())
```

```
Product IDs linked to multiple names: 0
Product names linked to multiple IDs: 0
```

Loads a fresh copy of the original Superstore dataset so we can compare it with the cleaned version.

```
raw_df = pd.read_csv("Sample - Superstore.csv", encoding="cp1252")
```

Compares the original and current dataset sizes to confirm whether rows or columns changed.

```
print("Original dataset:", raw_df.shape)
print("Current dataset:", df.shape)

print("Rows added/removed:", len(df) - len(raw_df))
print("Columns added:", df.shape[1] - raw_df.shape[1])
```

```
Original dataset: (9994, 21)
Current dataset: (9994, 22)
Rows added/removed: 0
Columns added: 1
```

Checks that customer IDs and customer names have a consistent one-to-one relationship.

```
customer_id_check = df.groupby("customer_id")["customer_name"].nunique()
customer_name_check = df.groupby("customer_name")["customer_id"].nunique()

print("Customer IDs linked to multiple names:",
      (customer_id_check > 1).sum())

print("Customer names linked to multiple IDs:",
      (customer_name_check > 1).sum())
```

```
*** Customer IDs linked to multiple names: 0
Customer names linked to multiple IDs: 0
```

✓ Confirms that every sub-category belongs to only one main category.

```
subcategory_check = df.groupby("sub_category")["category"].nunique()

print("Sub-Categories linked to multiple Categories:",
      (subcategory_check > 1).sum())
```

Sub-Categories linked to multiple Categories: 0

✓ Finds the earliest and latest order dates to understand the dataset's time period.

```
print("Earliest Order Date:", df["order_date"].min())
print("Latest Order Date:", df["order_date"].max())
```

Earliest Order Date: 2014-01-03 00:00:00
Latest Order Date: 2017-12-30 00:00:00

✓ Creates useful business features such as year, month, shipping time, profit margin, sales per unit, profit status, and discount groups.

```
df["order_year"] = df["order_date"].dt.year
df["order_month"] = df["order_date"].dt.month
df["month_name"] = df["order_date"].dt.month_name()
df["order_quarter"] = df["order_date"].dt.quarter
df["year_month"] = df["order_date"].dt.to_period("M").astype(str)
df["shipping_days"] = (df["ship_date"] - df["order_date"]).dt.days
df["profit_margin_pct"] = (df["profit"] / df["sales"]) * 100
df["sales_per_unit"] = df["sales"] / df["quantity"]
df["profit_status"] = np.where(
    df["profit"] >= 0,
    "Profit",
    "Loss"
)
df["discount_band"] = pd.cut(
    df["discount"],
    bins=[-0.01, 0, 0.2, 0.5, 1.0],
    labels=["No Discount", "Low", "Medium", "High"]
)
```

✓ Checks the dataset size again after adding the new features.

```
df.shape
```

(9994, 32)

df.isnull().sum()

row_id 0
order_id 0
order_date 0
ship_date 0
ship_mode 0
customer_id 0
customer_name 0
segment 0
country 0
city 0
state 0
postal_code 0
region 0
product_id 0
category 0
sub_category 0
product_name 0
sales 0
quantity 0
discount 0
profit 0
standard_product_id 0
order_year 0
order_month 0
month_name 0
order_quarter 0
year_month 0
shipping_days 0
profit_margin_pct 0
sales_per_unit 0
profit_status 0
discount band 0

- Reviews the final dataset structure, column types, and number of records.

```
[00] print("Final Shape:", df.shape)
✓ On df.info()
```

```
Final Shape: (9994, 32)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 32 columns):
#   Column                Non-Null Count  Dtype
---  -
0   row_id                9994 non-null   int64
1   order_id              9994 non-null   object
2   order_date            9994 non-null   datetime64[ns]
3   ship_date             9994 non-null   datetime64[ns]
4   ship_mode             9994 non-null   object
5   customer_id           9994 non-null   object
6   customer_name         9994 non-null   object
7   segment              9994 non-null   object
8   country               9994 non-null   object
9   city                 9994 non-null   object
10  state                9994 non-null   object
11  postal_code          9994 non-null   int64
12  region               9994 non-null   object
13  product_id           9994 non-null   object
14  category             9994 non-null   object
15  sub_category         9994 non-null   object
16  product_name         9994 non-null   object
17  sales                9994 non-null   float64
18  quantity             9994 non-null   int64
19  discount             9994 non-null   float64
20  profit               9994 non-null   float64
21  standard_product_id  9994 non-null   object
22  order_year           9994 non-null   int32
23  order_month          9994 non-null   int32
24  month_name           9994 non-null   object
25  order_quarter        9994 non-null   int32
26  year_month           9994 non-null   object
27  shipping_days        9994 non-null   int64
28  profit_margin_pct    9994 non-null   float64
29  sales_per_unit       9994 non-null   float64
30  profit_status        9994 non-null   object
31  discount_band        9994 non-null   category
dtypes: category(1), datetime64[ns](2), float64(5), int32(3), int64(4), object(17)
memory usage: 2.3+ MB
```

- Converts postal codes to text because they are identifiers rather than numerical measurements.

```
[01] df["postal_code"] = df["postal_code"].astype(str)
✓ On
```

- Runs a final quality check for rows, columns, missing values, duplicates, and product mapping problems.

```
[02] print("Rows:", df.shape[0])
✓ On print("Columns:", df.shape[1])
print("Missing Values:", df.isnull().sum().sum())
print("Duplicate Rows:", df.duplicated().sum())

print(
    "Product IDs linked to multiple names:",
    (df.groupby("standard_product_id")["product_name"].nunique() > 1).sum()
)

print(
    "Product names linked to multiple IDs:",
    (df.groupby("product_name")["standard_product_id"].nunique() > 1).sum()
)
```

```
Rows: 9994
Columns: 32
Missing Values: 0
Duplicate Rows: 0
Product IDs linked to multiple names: 0
Product names linked to multiple IDs: 0
```

- Saves the fully cleaned and prepared dataset as Sample - Superstore Cleaned.csv.

```
[03] df.to_csv("Sample - Superstore Cleaned.csv", index=False)
✓ On
```

[+ Code](#)